
Fundamentals of Database

IT403

Smart Clinic Database System

Final Project Report

Date: August 2026

Student Details:

CRN:

Name: Hanadi Abdullah Alshehri

ID: S230035637

Name: Sakinah Ali Almuahini

ID: S210033402

Name: Sarah hadi alyami

ID: S220002526

Name: Kadi Alharbi

ID: S230019734

Table of Contents

1. Introduction	3
2. Project Objectives.....	3
3. System Requirements Analysis	4
4. Database Design.....	5
5. Database Implementation	7
6. SQL Operations and Queries	12
7. Testing and Results.....	15
8. Database Features	16
9. Challenges and Solutions	17
10. Conclusion	18
11. References.....	19
Appendix B: Screenshots	20

1. Introduction

The **Smart Clinic Database System** is a comprehensive relational database management system designed to organize, manage, and improve the daily operations of a modern private clinic. With the increasing importance of digital transformation in the healthcare sector, effective management of medical information has become essential for improving service quality and operational efficiency.

This project provides a structured database solution for managing essential clinic information, including patient records, employee details, doctor and nurse information, appointments, treatments, prescriptions, medicine inventory, and payment transactions.

The main objective of this project is to design and implement an efficient, secure, and scalable database system using **MySQL**. By centralizing data and establishing well-defined relationships between database entities, the system reduces data redundancy, improves data consistency, minimizes manual errors, and enables faster access to accurate healthcare information.

The project applies important relational database concepts, including entity relationships, normalization, primary and foreign keys, SQL queries, views, and triggers, providing a reliable foundation that can be extended in the future to support additional healthcare services and technological improvements.

2. Project Objectives

The main objective of this project is to design and implement a reliable relational database system for managing the operations of a smart clinic. The project aims to organize healthcare-related information efficiently while applying essential and advanced database concepts.

The key objectives of the project are:

- To design a structured relational database that supports clinic management activities.
- To store and manage patient medical information in an organized and secure manner.
- To efficiently manage information related to doctors, nurses, receptionists, and other clinic employees.
- To manage patient appointments and track treatments effectively.
- To maintain accurate medicine inventory records and prescription history.
- To develop and execute complex SQL queries for data retrieval, analysis, and reporting.
- To implement advanced database features, including **Views, Triggers, Primary and Foreign Key constraints**, and **Referential Integrity** to ensure data consistency and reliability.

3. System Requirements Analysis

3.1 System Description

The Smart Clinic system serves as a central hub for various users, including medical professionals and administrative staff. The system is built around several core entities that interact to provide a seamless workflow.

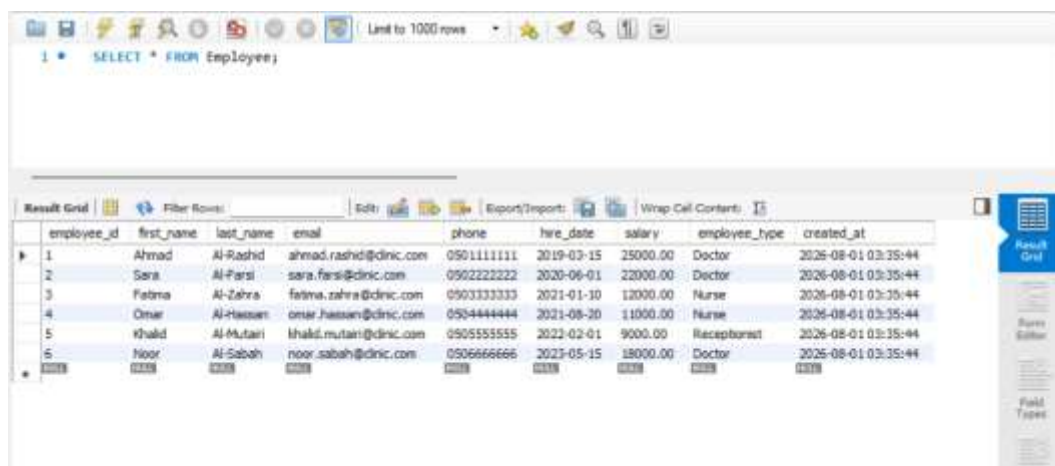
Entity	Description
Employee	Base information for all staff, specialized into Doctors, Nurses, and Receptionists.
Patient	Comprehensive records of patients including contact and medical details.
Appointment	Scheduling mechanism linking patients with specific doctors.
Treatment	Records of medical procedures performed during appointments.
Medicine	Inventory management for medications available in the clinic.
Prescription	Legal medical orders issued by doctors for patients.
Payment	Financial tracking for treatments and consultations.

5. Database Implementation

The database was implemented using MySQL. The script includes the creation of the database, tables, constraints, and the insertion of sample data to verify the system's functionality.

5.1 Tables Creation

Employee Table

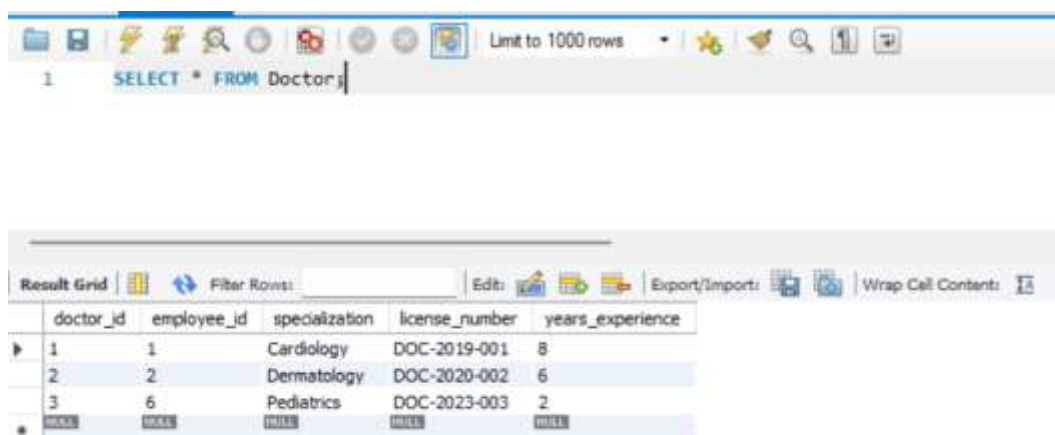


The screenshot shows the MySQL Workbench interface with the 'Employee' table selected. The SQL query is 'SELECT * FROM Employee;'. The result grid displays the following data:

employee_id	first_name	last_name	email	phone	hire_date	salary	employee_type	created_at
1	Ahmad	Al-Rashid	ahmad.rashid@clinic.com	0901111111	2019-03-15	25000.00	Doctor	2025-08-01 03:35:44
2	Sara	Al-Farsi	sara.farsi@clinic.com	0902222222	2020-06-01	22000.00	Doctor	2025-08-01 03:35:44
3	Fatma	Al-Zahra	fatma.zahra@clinic.com	0903333333	2021-01-10	12000.00	Nurse	2025-08-01 03:35:44
4	Omar	Al-Hassan	omar.hassan@clinic.com	0904444444	2021-08-20	11000.00	Nurse	2025-08-01 03:35:44
5	Khalid	Al-Mutairi	khalid.mutairi@clinic.com	0905555555	2022-02-01	9000.00	Receptionist	2025-08-01 03:35:44
6	Noor	Al-Sabah	noor.sabah@clinic.com	0906666666	2023-05-15	18000.00	Doctor	2025-08-01 03:35:44

Screenshot: Employee Table

Doctor Table

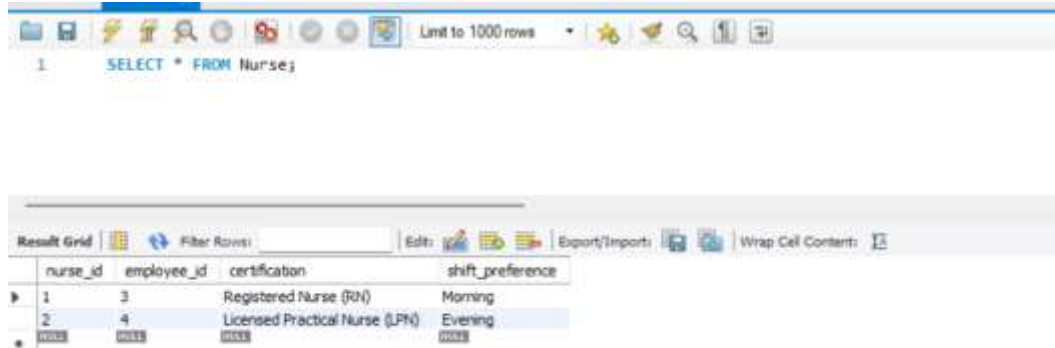


The screenshot shows the MySQL Workbench interface with the 'Doctor' table selected. The SQL query is 'SELECT * FROM Doctor;'. The result grid displays the following data:

doctor_id	employee_id	specialization	license_number	years_experience
1	1	Cardiology	DOC-2019-001	8
2	2	Dermatology	DOC-2020-002	6
3	6	Pediatrics	DOC-2023-003	2

Screenshot: Doctor Table

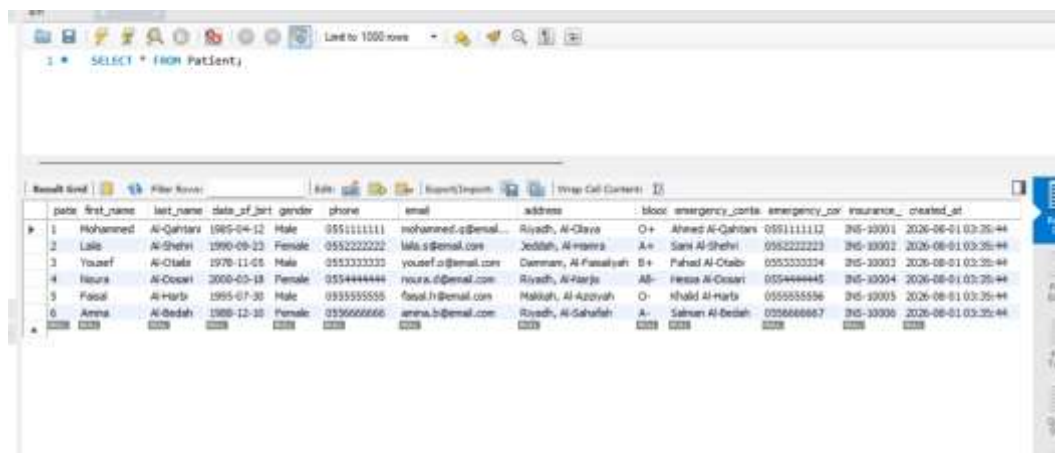
Nurse Table



nurse_id	employee_id	certification	shift_preference
1	3	Registered Nurse (RN)	Morning
2	4	Licensed Practical Nurse (LPN)	Evening

Screenshot: Nurse Table

Patient Table



patid	first_name	last_name	date_of_birth	gender	phone	email	address	blood	emergency_conta	emergency_cor	insurance_	created_at
1	Mohammed	Al-Qahera	1985-04-12	Male	0551111111	mohammed.q@email.com	Riyadh, Al-Qays	O+	Ahmed Al-Qahera	0551111112	SHS-10001	2026-06-01 03:35:44
2	Laila	Al-Shehri	1990-09-23	Female	0552222222	laila.s@email.com	Jeddah, Al-Masra	A+	Sami Al-Shehri	0552222223	SHS-10002	2026-06-01 03:35:44
3	Yousef	Al-Otaibi	1978-11-05	Male	0553333333	yousef.o@email.com	Dammam, Al-Fasalayah	B+	Pahad Al-Otaibi	0553333334	SHS-10003	2026-06-01 03:35:44
4	Noura	Al-Ossari	2000-03-18	Female	0554444444	noura.o@email.com	Riyadh, Al-Harji	AB-	Hessa Al-Ossari	0554444445	SHS-10004	2026-06-01 03:35:44
5	Faisal	Al-Harbi	1995-07-30	Male	0555555555	faisal.h@email.com	Makkah, Al-Azayyah	O-	Khalid Al-Harbi	0555555556	SHS-10005	2026-06-01 03:35:44
6	Amina	Al-Bedah	1988-12-30	Female	0556666666	amina.b@email.com	Riyadh, Al-Sahafah	A-	Sahar Al-Bedah	0556666667	SHS-10006	2026-06-01 03:35:44

Screenshot: Patient Table

Appointment Table

1 SELECT * FROM Appointments;

appointment_id	patient_id	doctor_id	appointment_date	status	reason	notes	created_at
1	1	1	2026-07-20 09:00:00	Completed	Chest pain and shortness of breath	Patient has history of hypertension	2026-08-01 03:35:44
2	2	2	2026-07-26 10:30:00	Completed	Skin rash on arms	Prescribed topical cream	2026-08-01 03:35:44
3	3	1	2026-07-28 14:00:00	Scheduled	Annual heart checkup	Bring previous ECG reports	2026-08-01 03:35:44
4	4	3	2026-07-29 11:00:00	Scheduled	Adult vaccination	Flu vaccine for adult patient	2026-08-01 03:35:44
5	5	2	2026-07-30 16:00:00	Cancelled	Acne treatment follow-up	Patient cancelled due to travel	2026-08-01 03:35:44
6	6	1	2026-07-31 08:30:00	Completed	High blood pressure monitoring	Blood pressure: 140/90	2026-08-01 03:35:44

Screenshot: Appointment Table

Treatment Table

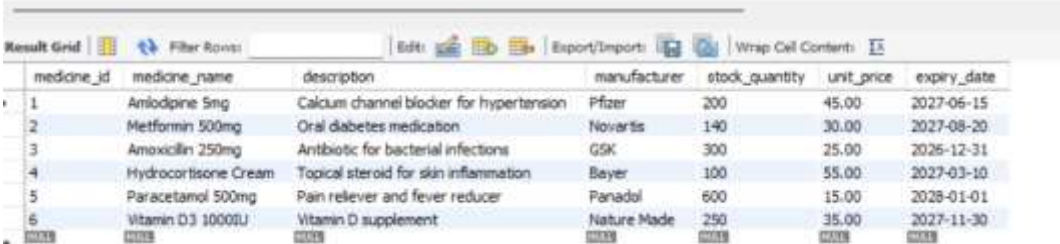
1 SELECT * FROM Treatment;

treatment_id	treatment_name	description	cost	duration_minutes
1	ECG Test	Electrocardiogram to check heart activity	150.00	30
2	Blood Test	Complete blood count and analysis	80.00	15
3	Skin Biopsy	Removal of skin sample for examination	300.00	45
4	Vaccination	Administration of vaccine	50.00	10
5	Physical Examination	General health checkup	100.00	20
NULL	NULL	NULL	NULL	NULL

Screenshot: Treatment Table

Medicine Table

```
1 SELECT * FROM Medicine;
```

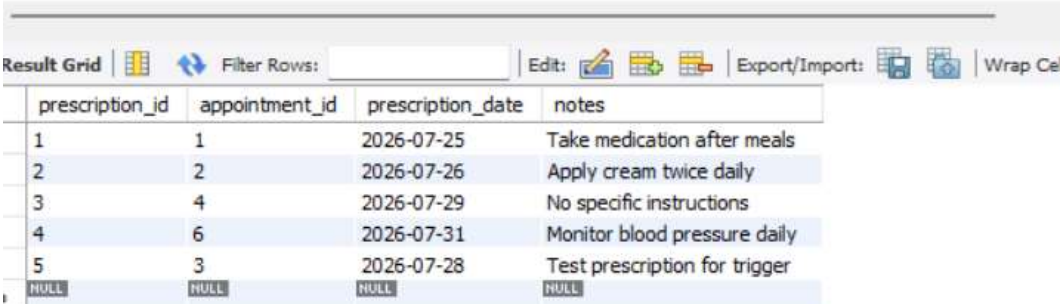


medicine_id	medicine_name	description	manufacturer	stock_quantity	unit_price	expiry_date
1	Amlodipine 5mg	Calcium channel blocker for hypertension	Pfizer	200	45.00	2027-06-15
2	Metformin 500mg	Oral diabetes medication	Novartis	140	30.00	2027-08-20
3	Amoxicillin 250mg	Antibiotic for bacterial infections	GSK	300	25.00	2026-12-31
4	Hydrocortisone Cream	Topical steroid for skin inflammation	Bayer	100	55.00	2027-03-10
5	Paracetamol 500mg	Pain reliever and fever reducer	Panadol	600	15.00	2028-01-01
6	Vitamin D3 1000IU	Vitamin D supplement	Nature Made	250	35.00	2027-11-30

Screenshot: Medicine Table

Prescription Table

```
1 SELECT * FROM Prescription;
```



prescription_id	appointment_id	prescription_date	notes
1	1	2026-07-25	Take medication after meals
2	2	2026-07-26	Apply cream twice daily
3	4	2026-07-29	No specific instructions
4	6	2026-07-31	Monitor blood pressure daily
5	3	2026-07-28	Test prescription for trigger
NULL	NULL	NULL	NULL

Screenshot: Prescription Table

Payment Table

```
1 SELECT * FROM Payment;
```

payment_id	patient_id	appointment_id	amount	payment_date	payment_method	status
1	1	1	230.00	2026-07-25	Card	Paid
2	2	2	300.00	2026-07-26	Cash	Paid
3	3	3	150.00	2026-07-28	Insurance	Pending
4	5	5	0.00	2026-07-30	Cash	Refunded
5	6	6	100.00	2026-07-31	Card	Paid
NULL	NULL	NULL	NULL	NULL	NULL	NULL

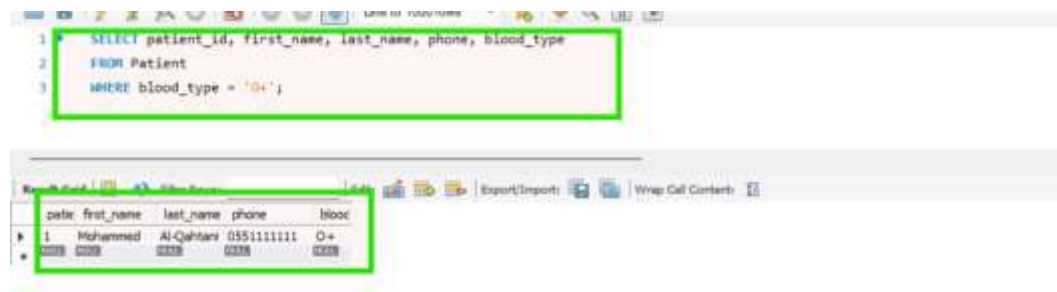
Screenshot: Payment Table

6. SQL Operations and Queries

To demonstrate the system's capability to handle real-world scenarios, several SQL operations were implemented, ranging from basic data retrieval to complex analytical queries.

Basic SELECT Query

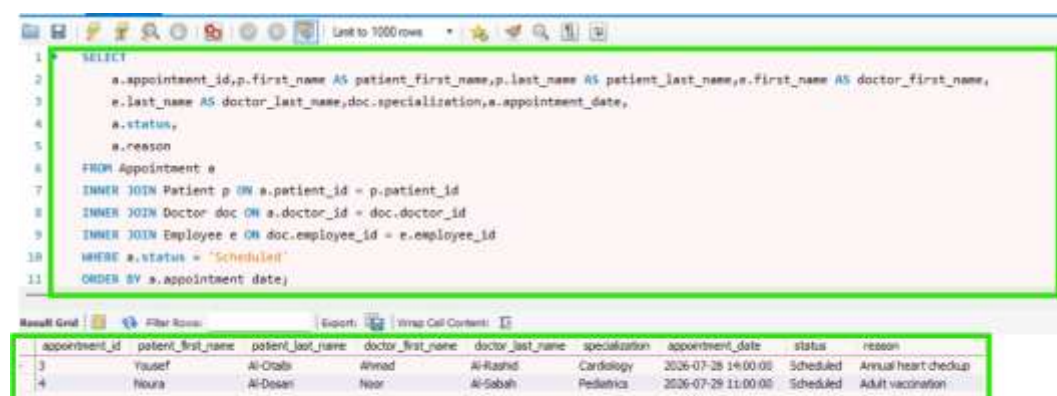
Used to retrieve all patient records.



Execution Result: Basic SELECT Query

JOIN Query

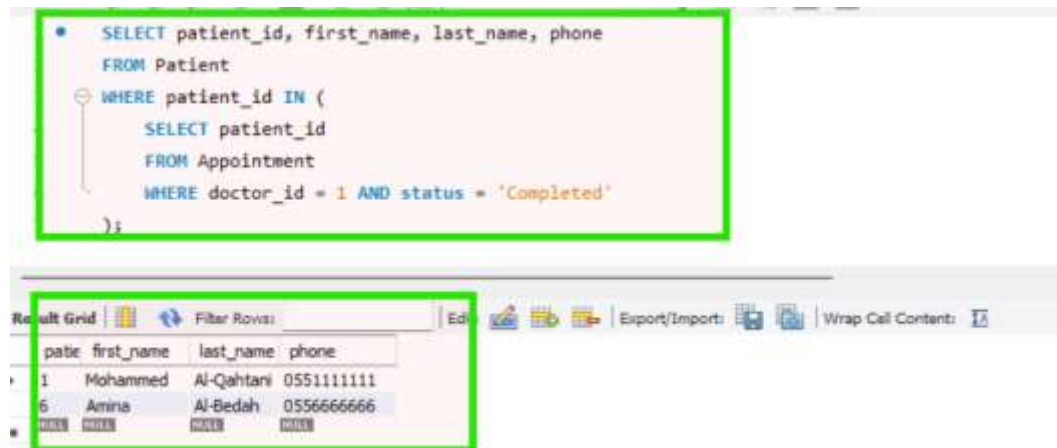
Combines Patient and Appointment data to show scheduled visits.



Execution Result: JOIN Query

Nested Query

Identifies doctors who have handled more than the average number of appointments.



The screenshot shows a SQL query in a text editor and its execution result in a table. The query is a nested query that selects patient details from the Patient table where the patient_id is in the list of patients who have completed appointments with doctor_id = 1.

```

SELECT patient_id, first_name, last_name, phone
FROM Patient
WHERE patient_id IN (
    SELECT patient_id
    FROM Appointment
    WHERE doctor_id = 1 AND status = 'Completed'
);

```

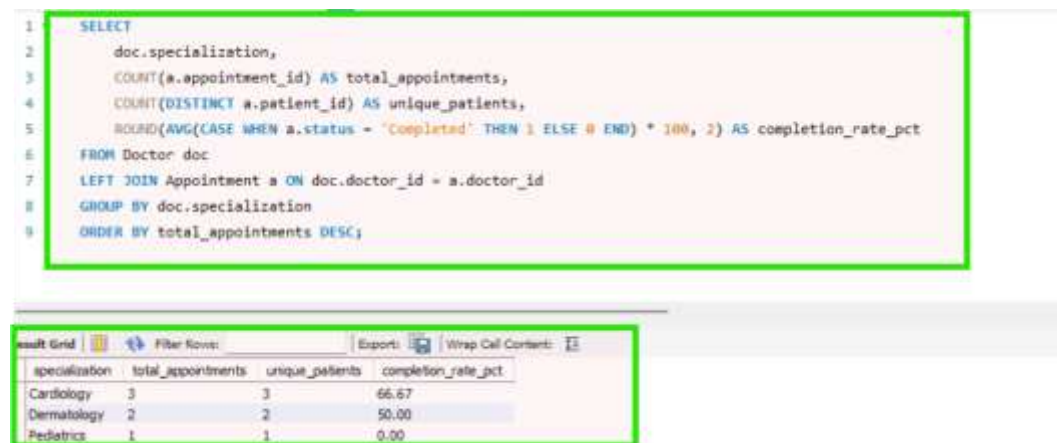
The execution result is a table with the following data:

patient_id	first_name	last_name	phone
1	Mohammed	Al-Qahtani	0551111111
6	Amina	Al-Bedah	0556666666

Execution Result: Nested Query

Aggregate Function

Calculates the total revenue generated from payments.



The screenshot shows a SQL query in a text editor and its execution result in a table. The query uses aggregate functions to calculate the total appointments, unique patients, and completion rate percentage for each doctor specialization.

```

SELECT
    doc.specialization,
    COUNT(a.appointment_id) AS total_appointments,
    COUNT(DISTINCT a.patient_id) AS unique_patients,
    ROUND(AVG(CASE WHEN a.status = 'Completed' THEN 1 ELSE 0 END) * 100, 2) AS completion_rate_pct
FROM Doctor doc
LEFT JOIN Appointment a ON doc.doctor_id = a.doctor_id
GROUP BY doc.specialization
ORDER BY total_appointments DESC;

```

The execution result is a table with the following data:

specialization	total_appointments	unique_patients	completion_rate_pct
Cardiology	3	3	66.67
Dermatology	2	2	50.00
Pediatrics	1	1	0.00

Execution Result: Aggregate Function

UPDATE Operation

Modifies employee salary or patient contact details.

```

1  -- SELECT
2  * SELECT medicine_id, medicine_name, stock_quantity
3    FROM Medicine
4   WHERE medicine_id = 5;
5  -- UPDATE
6  * UPDATE Medicine
7    SET stock_quantity = stock_quantity + 100
8   WHERE medicine_id = 5;
9  -- SELECT
10 * SELECT medicine_id, medicine_name, stock_quantity
11    FROM Medicine
12   WHERE medicine_id = 5;

```

medicine_id	medicine_name	stock_quantity
5	Paracetamol 500mg	800

Execution Result: UPDATE Operation

VIEW Implementation

A virtual table providing a summary of daily clinic activities.

```

1  CREATE OR REPLACE VIEW Patient_Appointment_Summary AS
2  SELECT
3    p.patient_id,
4    CONCAT(p.first_name, ' ', p.last_name) AS full_name, p.phone, p.blood_type,
5    COUNT(DISTINCT a.appointment_id) AS total_appointments,
6    COUNT(DISTINCT CASE WHEN a.status = 'Completed' THEN a.appointment_id END) AS completed_appointments, COALESCE(SUM(py.amount), 0) AS total_paid_amount,
7    MAX(a.appointment_date) AS last_appointment_date
8  FROM Patient p
9  LEFT JOIN Appointment a ON p.patient_id = a.patient_id
10 LEFT JOIN Payment py ON p.patient_id = py.patient_id AND py.status = 'Paid'
11 GROUP BY p.patient_id, p.first_name, p.last_name, p.phone, p.blood_type;

```

patr	full_name	phone	blood	total_appointments	completed_appointments	total_paid_amount	last_appointment_date
1	Mohammed Al-Qatani	5551111111	O+	1	1	230.00	2026-07-25 09:00:00
2	Laila Al-Shaykh	5552222222	A+	1	1	300.00	2026-07-26 09:30:00
3	Youssef Al-Chabi	5553333333	B+	0	0	0.00	2026-07-28 14:00:00
4	Youssef Al-Ozairi	5554444444	AB-	0	0	0.00	2026-07-29 11:00:00
5	Faisal Al-Harbi	5555555555	O-	0	0	0.00	2026-07-30 18:00:00
6	Amr Al-Badri	5556666666	A-	1	1	100.00	2026-07-31 08:30:00

Execution Result: VIEW Implementation

7. Testing and Results

The Smart Clinic Database System underwent comprehensive testing to ensure data integrity, accuracy, and reliable performance. All SQL scripts were executed and verified using **MySQL Workbench**, and the obtained results were compared with the predefined system requirements.

The testing process included several important areas:

1. Referential Integrity Testing

The database relationships and constraints were tested to ensure data consistency. Foreign key constraints were verified to prevent invalid operations, such as deleting a patient record that is associated with existing active appointments. This ensured that related data remains accurate and protected from unintended deletion.

2. Trigger Testing

The implemented database trigger was tested to verify automatic updates in the medicine inventory. The test confirmed that medicine stock quantities are automatically reduced when a new prescription detail is added, ensuring accurate inventory management without requiring manual updates.

3. SQL Query Testing

All implemented SQL operations, including SELECT queries, JOIN queries, nested queries, aggregate functions, UPDATE, DELETE, views, and triggers, were executed successfully. The results confirmed that the database can efficiently retrieve, modify, and manage clinic information according to the project requirements.

Overall, the testing results demonstrated that the Smart Clinic Database System satisfies the required functionality and maintains data integrity through proper database design and implementation.

8. Database Features

The Smart Clinic Database System provides several important features that improve reliability, efficiency, and maintainability.

1. Data Integrity

Data integrity is maintained through the implementation of **Primary Keys**, **Foreign Keys**, and **NOT NULL constraints**. These constraints ensure that data remains accurate, consistent, and properly connected between related tables.

2. Scalability

The database structure follows normalization principles, allowing the system to be easily expanded in the future. New features, such as additional medical departments, healthcare services, or employee roles, can be added without significant changes to the existing design.

3. Automation

Database automation is achieved through the use of **Triggers**. For example, the implemented trigger automatically updates medicine stock levels when prescription details are added, reducing manual work and improving accuracy.

4. Security and Access Control

The system design supports future implementation of role-based access control. Different permissions can be assigned based on employee classifications, such as doctors, nurses, and receptionists, ensuring that users can access only the information required for their roles.

9. Challenges and Solutions

Challenges and Solutions

Challenge 1: Designing the Employee Specialization Hierarchy

Challenge:

One of the main challenges was designing the specialization hierarchy for employees, as the system includes different employee types such as doctors, nurses, and receptionists, while sharing common employee information.

Solution:

An Enhanced Entity-Relationship (EER) model was implemented using a superclass/subclass approach. The common attributes were stored in the **Employee** superclass, while role-specific attributes were managed in separate tables for **Doctor**, **Nurse**, and **Receptionist**. This design reduced data redundancy and improved database organization.

Challenge 2: Managing the Many-to-Many Relationship Between Treatments and Appointments

Challenge:

Another challenge was handling the many-to-many relationship between appointments and treatments, where one appointment may include multiple treatments and one treatment can be associated with multiple appointments.

Solution:

A junction table named **Appointment_Treatment** was created to resolve this many-to-many relationship. This approach maintained database normalization up to the Third Normal Form (3NF), improved data integrity, and ensured efficient relationship management between appointments and treatments.

10. Conclusion

The Smart Clinic Database System project successfully achieved its objective of designing and implementing a reliable relational database solution for healthcare management. The system provides an organized approach for managing essential clinic operations, including employee management, patient records, appointments, treatments, prescriptions, medicines, and payments.

By applying fundamental and advanced database concepts, such as entity relationships, primary and foreign keys, SQL queries, views, and triggers, the project demonstrated the practical application of database design and implementation principles.

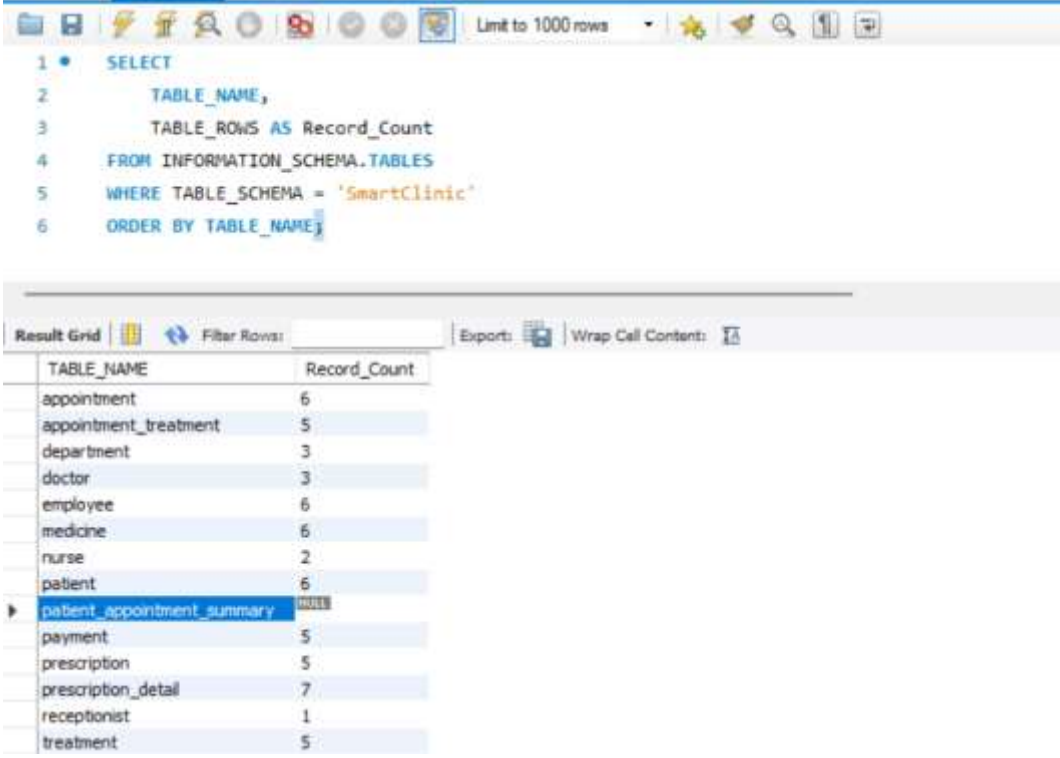
This project provided valuable practical experience in database modeling, MySQL implementation, query development, system testing, and technical documentation. The developed system can serve as a foundation for future improvements and can be extended to support additional healthcare features and modern technological requirements.

11. References

1. Oracle. (2026). **MySQL Documentation**.
Available at: <https://dev.mysql.com/doc/>
2. Oracle. (2026). **MySQL Workbench Documentation**.
Available at: <https://dev.mysql.com/doc/workbench/en/>
3. Elmasri, R., & Navathe, S. B. (2015). **Fundamentals of Database Systems** (7th Edition). Pearson.
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. **Database System Concepts**. McGraw-Hill.
5. IT403 Database Course Materials, King Saud University.
6. Git Documentation.
Available at: <https://git-scm.com/doc>
7. GitHub Repository – **Smart Clinic Database System Project**.
Available at: <https://github.com/anyanymouse/SmartClinic-Database>

Appendix B: Screenshots

This appendix contains additional evidence of the system implementation.



The screenshot shows a SQL query editor with a query that selects table names and row counts from the 'SmartClinic' schema. Below the query, a 'Result Grid' displays the output as a table with two columns: 'TABLE_NAME' and 'Record_Count'. The results list various tables and their corresponding row counts, with 'patient_appointment_summary' highlighted.

```
1 • SELECT
2     TABLE_NAME,
3     TABLE_ROWS AS Record_Count
4 FROM INFORMATION_SCHEMA.TABLES
5 WHERE TABLE_SCHEMA = 'SmartClinic'
6 ORDER BY TABLE_NAME
```

TABLE_NAME	Record_Count
appointment	6
appointment_treatment	5
department	3
doctor	3
employee	6
medicine	6
nurse	2
patient	6
patient_appointment_summary	NULL
payment	5
prescription	5
prescription_detail	7
receptionist	1
treatment	5

Figure: Verification of Data Insertion